

Python Programs – Questions 1 to 10

Simple, exam-friendly codes based on the uploaded question list

Q1. Student Performance Analysis — List + Tuple

Concept: Uses a list for marks and a tuple for storing the final result and number of subjects.

Python Code:

```
# Student marks analysis
marks = [78, 92, 65, 88, 95]

print("Marks:", marks)
print("Highest:", max(marks))
print("Lowest:", min(marks))
print("Average:", sum(marks) / len(marks))

result = ("Pass" if min(marks) >= 35 else "Fail", len(marks))
print("Result tuple:", result)
```

Q2. Customer Name Analysis — String + List

Concept: Uses strings for name operations and a list to store and filter customer names.

Python Code:

```
# Customer name analysis
names = ["Ravi", "Priya", "Arun", "Pujitha", "Sita"]

search = input("Enter customer name: ").strip()

print("Total customers:", len(names))
print("Names starting with A:", [n for n in names if n.startswith("A")])
print("Name lengths:", [len(n) for n in names])

if search in names:
    print("Customer found")
else:
    print("Customer not found")
```

Q3. Product Inventory Analysis — List + Tuple

Concept: Each product is stored as a tuple (name, stock), and all product tuples are stored in a list.

Python Code:

```
# Product inventory analysis
products = [("Pen", 20), ("Book", 10), ("Bag", 5), ("Pencil", 30)]

print("Products with stock:")
for name, stock in products:
    print(name, ":", stock)

total = sum(stock for name, stock in products)
low_stock = [name for name, stock in products if stock < 10]

print("Total stock:", total)
print("Low-stock products:", low_stock)
```

Q4. Text Analysis — String + List

Concept: Converts the input string into a list of words and performs basic text analysis.

Python Code:

```
# Text analysis
text = input("Enter a sentence: ")

words = text.lower().split()

print("Words:", words)
print("Number of words:", len(words))
print("Longest word:", max(words, key=len) if words else "")
print("Words with length > 4:", [w for w in words if len(w) > 4])
```

Q5. Employee Skill Analysis — Tuple + List + String

Concept: Employee data is represented using tuples, skills are stored in lists, and strings are used for skill matching.

Python Code:

```
# Employee skill analysis
employees = [
    ("Ravi", ["Python", "SQL"]),
    ("Priya", ["Java", "Python"]),
    ("Arun", ["C", "SQL"])
]

skill = input("Enter skill: ").strip().lower()

for name, skills in employees:
    if skill in [s.lower() for s in skills]:
        print(name, "has", skill)

print("Employee data:")
for emp in employees:
    print(emp)
```

Q6. Transaction Analysis — Tuple + List

Concept: Transactions are stored as tuples inside a list, allowing total, maximum, minimum and filtering operations.

Python Code:

```
# Transaction analysis
transactions = [
    ("T101", 500),
    ("T102", 1200),
    ("T103", 300),
    ("T104", 1500)
]

amounts = [amount for tid, amount in transactions]

print("Total amount:", sum(amounts))
print("Highest transaction:", max(amounts))
print("Lowest transaction:", min(amounts))

large = [t for t in transactions if t[1] > 1000]
print("Transactions above 1000:", large)
```

Q7. Password Security Analysis — String + List

Concept: Checks password length, uppercase, lowercase, digit and special character using string methods and a list.

Python Code:

```
# Password security analysis
password = input("Enter password: ")

checks = [
    len(password) >= 8,
    any(c.isupper() for c in password),
    any(c.islower() for c in password),
    any(c.isdigit() for c in password),
    any(not c.isalnum() for c in password)
]

print("Length >= 8:", checks[0])
print("Uppercase:", checks[1])
print("Lowercase:", checks[2])
print("Digit:", checks[3])
print("Special character:", checks[4])

print("Strong password" if all(checks) else "Weak password")
```

Q8. Code Analysis and Debugging — List + Tuple + String

Concept: This example demonstrates correct iteration over tuples in a list, conditional analysis, list creation and string joining.

Python Code:

```
# Corrected code-analysis example
data = [("Python", 90), ("Java", 80), ("SQL", 85)]

for subject, mark in data:
    if mark >= 85:
        print(subject, ":", "Excellent")
    elif mark >= 50:
        print(subject, ":", "Pass")
    else:
        print(subject, ":", "Fail")

subjects = [item[0] for item in data]
print("Subjects:", subjects)
print("Joined:", " ", ".join(subjects))
```


Q9. Word Frequency and Data Analysis — String + List + Tuple

Concept: Splits a string into a list of words and stores each word-frequency pair as tuples in a list.

Python Code:

```
# Word frequency analysis
text = input("Enter text: ").lower()
words = text.split()

frequency = {}
for word in words:
    frequency[word] = frequency.get(word, 0) + 1

print("Word frequency:")
for word, count in frequency.items():
    print((word, count))

result = list(frequency.items())
print("Frequency tuples:", result)
```

Q10. Integrated Student Data Analysis — String + List + Tuple

Concept: Combines strings, lists and tuples in one program to search a student and calculate total, average and grade.

Python Code:

```
# Integrated student data analysis
students = [
    ("Ravi", [80, 75, 90]),
    ("Priya", [95, 88, 92]),
    ("Arun", [60, 70, 65])
]

name = input("Enter student name: ").strip()

for student, marks in students:
    if student.lower() == name.lower():
        total = sum(marks)
        average = total / len(marks)
        print("Name:", student)
        print("Marks:", marks)
        print("Total:", total)
        print("Average:", average)
        print("Grade:", "A" if average >= 80 else "B" if average >= 60 else "C")
        break
    else:
        print("Student not found")
```